FISH SHOPPING

Catch. Process. Serve. Expand.

FULL 3D MOBILE & WEB GAME BLUEPRINT

An original fishing-business simulation: walk to the water, catch fish, carry them to processing machines, stock sales counters, serve customers, hire workers, and grow a waterfront business.

Platform	Chosen implementation
Android mobile	Kotlin + Jetpack Compose + Filament
Web	TypeScript + React + Three.js / WebGL 2
Common product contract	Shared content schemas, balance data, assets, and parity fixtures

Prepared for Willio Jeanvilma | November 5, 2026 | Version 1.0

Project title: Fish Shopping. Platform scope: native Android plus desktop and touch-browser web. An iOS release requires an additional native integration project. This document defines the intended product; budgets, balance values, counts, and schedules are proposals to validate. It does not claim a completed implementation.

Product definition and experience

The player physically operates a growing 3D seafood business. A character walks between a fishing pier, processing machines, storage, counters, and upgrade areas. Stacks of fish and product boxes move visibly through the world. Customers arrive, queue, buy, and leave. Earnings unlock a larger, more automated operation.

Attribute	Product decision
Genre	Arcade business simulation with active and idle progression
Presentation	Fully 3D stylized world with polished native/HTML interface overlays
View	Elevated third-person camera; compact readable work areas
Session	3-10 minute mobile visits; longer expansion sessions
Primary goal	Grow a small pier stall into a regional seafood operation
Launch mode	Single-player, offline-capable; optional account synchronization
Player agency	Movement, fishing, transport, station allocation, upgrades, staff priorities

Design pillars

Visible production: every sale can be traced to catching, processing, and stocking. **Constant useful choices:** decide which bottleneck to fix next. **Physical satisfaction:** responsive movement, tidy item transfer, smooth machine animation, and clear money feedback. **Fair progression:** meaningful upgrades without forcing payment or advertisements.

What fully 3D means

Characters, water, buildings, fish, machines, shelves, and customers are rendered as 3D objects. Menus and HUD remain readable interface layers. Real-time lighting and camera movement support depth, while stylized assets keep mobile performance attainable. UI panels are functional screens, not baked images.

Complete gameplay loop

Step	Interaction and result
1. Catch	Stand in a valid fishing area; cast automatically or press the action button.
2. Collect	Caught fish enter available carrying slots, with a visible stack.
3. Deliver	Approach a machine input zone to deposit compatible raw fish.
4. Process	A timed recipe converts reserved inputs into packaged products.
5. Stock	Carry finished products to the matching sales counter.
6. Sell	Customers collect stock, queue, pay, and exit.
7. Reinvest	Improve tools, capacity, speed, storage, staff, and business area.
8. Expand	Unlock species, processing lines, contracts, and new harbors.

Active and automated play

Early play is hands-on. A fisher later supplies raw stock, a porter moves goods, an operator supports processing, and a cashier handles transactions. The player remains useful by catching rarer fish, filling urgent gaps, fulfilling orders, and making expansion decisions.

Bottleneck examples

If raw stock piles up, processing is too slow. If outputs fill but counters are empty, transport is constrained. If customers leave in a queue, checkout is constrained. Surface these causes through a concise business panel rather than expecting players to infer them from profit alone.

First-session promise

Within five minutes the player should catch, process, stock, receive payment, and purchase one upgrade. Within fifteen minutes, the player should unlock a helper or a second product line. These are tuning targets, not guarantees.

World layout and harbor progression

Each harbor is a compact authored level with distinct fishing zones and a business footprint. Use modular expansion plots instead of a seamless open world. A map screen switches harbors after saving the current operation.

Harbor	Identity and progression
Sunrise Pier	Tutorial harbor: common fish, basic cutter, one stall, simple staff.
Coral Bay	Additional species, preparation station, larger customer mix.
Stormbreak Dock	Higher-capacity catches, chilled storage, bulk orders.
Northlight Harbor	Premium cold-water products, advanced machines, seasonal styling.
Grand Marina	Several production chains, premium visitors, long-term mastery objectives.

Layout rules

Fishing access sits near raw storage. Machines connect to output collection lanes. Customer routes remain separate from service lanes where possible. Every expansion keeps an accessible path between spawn, core stations, shop, and pier. Prevent decoration placement from blocking navigation.

Plots and construction

Walk into a marked plot and inspect its price, purpose, dependencies, and expected output. Confirm spending before construction. Use a brief build animation; opening a station registers its inventory, recipe, input zone, and worker task providers atomically.

Cross-harbor policy

Only the active harbor runs full visual simulation. Inactive harbors use the same constrained production approximation as offline earnings. Business inventories remain separate; explicit shipping orders may transfer goods later. Global coins, profile, and unlocks remain shared.

Movement, camera, and interaction

Mobile controls

Default to landscape orientation with a left floating joystick and a contextual right action button. Offer tap-to-move as an accessibility option. Automatically transfer goods when the character stays within a station zone; show the intended transfer and a cancel path before expensive or irreversible actions.

Web controls

WASD or arrow keys move; E performs the contextual action. Pointer or touch controls activate menus. Touch-browser devices receive the same virtual controls as mobile. Capture gameplay input only when the game has focus; text inputs and menus must never move the character.

Camera behavior

Use a follow camera with eased movement, limited zoom, and an occlusion fade for selected scenery. Lock rotation by default to preserve directional clarity; optional rotation is available on desktop. Avoid camera motion during precise interactions and allow reduced-motion settings.

Interaction type	Execution contract
Automatic transfer	Requires correct item, free capacity, proximity, and transfer cooldown.
Fishing	Requires valid zone, available carry capacity, and no blocking menu.
Purchase	Shows price and result; validates balance and prerequisites at commit.
Machine interaction	Shows input, output, rate, capacity, and blocked reason.
Staff assignment	Select role and work zone; confirm a reachable assignment.

Collision and navigation

Use a 2D ground-plane navigation graph with 3D presentation. Characters collide with station footprints and level obstacles; do not simulate every carried fish as a rigid body. Movement slowdown from inventory is optional and must remain mild enough to avoid frustrating errands.

Fishing system and species

Fishing flow

The player enters a pier zone, casts, waits through a short anticipation animation, and retrieves a catch. Default play uses automatic timing while standing in the zone. An optional timing bonus improves the next catch slightly without making accessibility assistance unprofitable.

Species example	Game role
Silver sprat	Starter catch; quick cycle; processed into basic portions.
Blue mackerel	Early upgrade fish; greater value and slightly longer catch time.
Reef snapper	Midgame product for fresh fillets and mixed orders.
Harbor crab	Special trap zone and separate preparation recipe.
Deepwater tuna	Large catch; higher carry requirement and premium products.
North salmon	Advanced harbor input for chilled premium portions.
Golden catch	Rare collectible visual variant; fixed bonus without paid odds.

Catch definition

Each species declares ID, harbor zones, unlock requirement, catch duration, carry units, recipe eligibility, base value, rarity weight, and 3D asset. Catch weights are game abstractions rather than biological claims. Show newly unlocked species in a collection book.

Tools

Rod power unlocks target categories; catch speed lowers cycle time; line control improves bonus timing; carry capacity increases trips. Cap combined speed reductions so animations and pacing remain readable. Do not require consumable bait for the basic progression loop.

Stock availability

Fish spawn from a zone resource meter that replenishes predictably. Avoid draining the tutorial zone completely. Special zones can have cooldowns and visible forecasts. Rare catches never gate mandatory early tasks.

Carrying, storage, and item ownership

Inventory is authoritative data; visible stacks are a pooled presentation. A stack changes immediately when ownership transfers, while short flying-item animations communicate the transition. Cosmetic animations must not grant or remove inventory twice.

Container	Required behavior
Player bag	Limited carry units; sorted by type; optional product priority.
Raw storage	Holds species inputs; supplies operators and permitted machines.
Machine input	Owns deposited ingredients; reports missing or incompatible materials.
Machine output	Holds finished batches; blocks production when full.
Cold storage	Holds premium finished products and feeds assigned porters.
Counter stock	Reserved for customer purchases; cannot be sold twice.

Transfer rule

Move min(source available, destination free capacity, transfer rate × elapsed time) whole units. Execute a transfer as one state mutation or transaction. Reject wrong item types without changing either inventory. Use explicit reservations when a worker or customer is travelling to collect stock.

Carry policy

Raw fish and packaged products share capacity measured in carry units. Large fish can occupy multiple units. The HUD shows capacity and item mix. Provide a simple drop-back-to-storage action so an inconvenient mixture cannot trap the player.

Freshness

Launch with no real-time spoilage while the app is closed. Optional advanced freshness operates only during active play, pauses in menus, and provides clear warnings and an ordinary recovery route. Keep the base game relaxing and consistent across platforms.

Machines and production recipes

Machine	Input and output role
Basic cutter	1 common fish → 2 basic portions.
Fillet station	1 eligible medium fish → 3 fillet packs.
Preparation bench	2 crab units → 3 prepared seafood boxes.
Chilling line	2 premium portions → 2 chilled packs.
Packing line	4 eligible packs → 1 bulk carton.
Premium kitchen	Optional later chain combining compatible seafood into a premium product.

Production state machine

Idle → waiting for input → reserve ingredients → processing → output ready → transfer output → repeat. A machine can also be blocked by output capacity or paused by the player. All blocked states show a concise reason and a corrective action.

Commit semantics

At batch start, remove the required input and record it in an in-progress batch. At completion, create output once using a unique batch ID. Save remaining duration and reserved materials. On load, resume the batch; never recreate both inputs and outputs. No automatic recipe switch discards reserved materials.

Upgrades

Speed reduces batch time; hopper capacity stores more input; output capacity reduces blocking; parallel lanes add throughput. Preview exact before/after values and the current limiting factor. The machine is a visible 3D object with conveyors, indicator lights, moving parts, and output stacks.

Recipe consistency

A recipe lists compatible species, required units, output units, duration, machine class, and unlock. Keep whole-number inventories and integer coin prices. Species substitution uses an explicit priority list so workers cannot consume a premium contract item unexpectedly.

Customers, counters, and checkout

Customer journey

Spawn → select an available offer → walk to counter → reserve stock → collect → join checkout → pay → exit. A customer can leave if no compatible stock is found or patience expires. Release all reservations on cancellation, despawn, save recovery, or path failure.

Customer type	Behavior
Local shopper	Small baskets and predictable demand.
Family buyer	Larger baskets; moderate patience.
Restaurant buyer	Scheduled bulk order with clear requirements.
Premium visitor	Requests a high-tier product and pays its listed premium price.
Rush group	Short event that temporarily increases arrivals within NPC budgets.

Demand and prices

Counters have a product assignment, stock capacity, displayed unit price, and queue anchor. Start with fixed content prices for stable balance. Later pricing experiments can offer a small bounded choice with visible demand effects, rather than an opaque economic simulation.

Checkout accounting

Move collected products into a customer basket. Payment atomically consumes the basket and credits coins once using a sale ID. Display floating coin feedback and update business metrics from the committed sale. Staff or player presence accelerates checkout according to defined rules.

Queue failure handling

Queue slots are reserved; waiting customers move forward without overlapping. After a navigation timeout, retry a reachable point, then exit and release goods according to the abandonment policy. Purchased goods are never returned after payment. Unpaid abandoned baskets return to stock or designated returns inventory.

Workers, automation, and routing

Role	Task and priority
Fisher	Supplies assigned raw species while carry and storage capacity permit.
Porter	Moves goods from production outputs or storage to compatible destinations.
Operator	Loads assigned machines and supports processing cycles.
Cashier	Serves the assigned checkout when customers are waiting.
Supervisor	Unlocks scheduling controls and aggregates bottleneck information.

Task scheduler

Generate tasks from unmet needs. Score urgency, path distance, demand, and assignment. A worker claims one task with a lease, reserves goods and destination capacity, walks the route, commits transfer, and releases the claim. Expired or invalid claims return to the pool.

Priority examples

Stock an empty popular counter before topping up an already full one. Finish a nearly complete contract only when the player enables contract priority. Avoid cycling goods between storage locations. A destination accepts a given product through a single configured policy.

Worker upgrades

Movement speed, carry units, task efficiency, and role training. Wage design uses an upfront hire cost and optional operating cost deducted only from earned business revenue, never negative debt that blocks casual play. Staff cannot spend coins or upgrades autonomously.

Assignment UI

Select a worker, choose role, assign stations or zones, and inspect their current task. Show reasons such as no route, no stock, full destination, or waiting customer. Reassignments safely cancel reservations after handling any goods already carried.

Economy and numerical balance

Use coins as the main currency. Optional cosmetic tokens can be earned through achievements. Keep early progression independent of premium currency. Upgrade costs are content-driven, visible, and validated before purchase.

Starter line worked example

A catch takes 4 seconds: 15 fish/minute before travel. The cutter consumes 1 fish every 5 seconds: 12 fish/minute and 24 portions/minute. Each portion sells for 5 coins. With enough transport, demand, and checkout capacity, revenue is capped at 120 coins/minute. Travel and idle intervals reduce realized revenue.

A cutter upgrade to 4 seconds supports 15 fish/minute and 30 portions/minute, increasing the theoretical line cap to 150 coins/minute. A 300-coin upgrade has a theoretical 10-minute incremental payback at +30 coins/minute. If fishing, transport, or demand is already limiting, realized payback takes longer.

Formula	Meaning
Line output/min	Minimum capacity across fishing, processing, transport, shelf supply, demand, checkout.
Gross revenue	Sum of committed sale quantity × unit price.
Net earnings	Revenue minus explicit operating costs.
Upgrade cost	Rounded base cost × growth factor raised to current level.
Production rate	Output units × parallel lanes × 60 / batch seconds.
Realized efficiency	Actual output / configured maximum output over an interval.

Economy safeguards

Use integer coins and quantities. Kotlin Long values must stay within the safe numeric range chosen for TypeScript; cap stored values below 2^53 and validate multiplication. Display compact notation without changing stored precision. Clamp offline durations and prevent negative inventory or repeated reward grants.

Progression, quests, and contracts

System	Design
Tutorial tasks	Teach catch, process, stock, payment, upgrade, and hiring.
Milestone quests	Unlock stations after visible cumulative achievements.
Daily-style objectives	Optional rotating goals; no mandatory streak penalty.
Bulk contracts	Reserve a known quantity and deliver for a displayed reward.
Species collection	Record first catches and grant fixed milestone rewards.
Harbor mastery	Earn stars for service quality, production variety, and completed goals.

Unlock graph

Starter fish → cutter → counter → first sales → capacity upgrade → porter → second recipe → cashier → bulk contract → next harbor. Every node declares prerequisites, cost, reward, and UI destination. Validate the graph for cycles and impossible resource requirements.

Contracts

A contract declares requested products, quantity, available duration, payout, and destination. Time-sensitive contracts count down only under the announced policy; default casual contracts have no real-world expiry. Player acceptance reserves no goods until explicit packing, preventing workers from silently consuming inventory.

Progression pacing

Aim for frequent early improvements, then larger strategic upgrades. The next useful target must always be visible. Use the bottleneck panel to recommend a reasoned upgrade, not simply the cheapest. Do not require rare random catches, payments, or calendar waiting to complete a harbor.

Recovery

If inventory is empty and coins are spent, basic fishing and starter production remain free. A reset option previews consequences and requires confirmation. Guest progress is valid and can later attach to an account.

Offline progress and time handling

Scope

Offline earnings begin only after enough staff are hired to operate a configured line. Manual actions do not happen while absent. Start with a four-hour cap and no paid requirement to claim the base reward. Show elapsed time, credited time, products sold, coins earned, and any blocked line.

Simulation policy

Use an integer, coarse simulation of assigned automated chains. Carry machine batches, inventories, storage limits, catch limits, and demand caps forward in bounded time steps. For cyclic or unsupported chains, stop at a validated boundary and report the restriction rather than minting unconstrained income.

Subtract already-accounted active time. At claim, atomically apply the calculated result and advance the accounted-through timestamp. Reopening the claim dialog or retrying a network request must return the same receipt rather than grant again.

Clock rules

For local-only casual saves, device time can be changed; cap duration, reject negative intervals, and accept that local rewards are editable. Online accounts use server timestamps and versioned claims. Do not advertise tamper-proof progress for an offline client.

Lifecycle behavior

Pause active simulation in background. Save before suspension where possible, also checkpoint periodically. On resume, reconcile elapsed time once and present the result before resuming movement. Account for native process death and browser tab eviction. Inactive harbors follow the same bounded model.

Screen inventory and navigation

Screen	Purpose and required content
Launch / loading	Progress, asset download state, retry, and unsupported-device message.
Welcome / profile	Continue guest, optional sign-in, recover cloud progress.
Main 3D harbor	World, player, stations, customers, HUD, joystick or keyboard hints.
Station detail	Recipe, inputs, outputs, rate, capacity, blockage, upgrade.
Upgrade shop	Categories, prices, prerequisites, before/after metrics.
Workers	Roster, roles, assignments, task status, training.
Warehouse	Product quantities, reservations, capacities, transfer priorities.
Contracts / quests	Objectives, progress, rewards, tracked task.
Harbor map	Unlocked locations, mastery, costs, production summaries.
Species book	Collection, zones, recipes, unlock information.
Business dashboard	Revenue, throughput, lost sales, bottleneck, history.
Offline summary	Elapsed time, cap, line output, claim receipt.
Settings / help	Audio, graphics, accessibility, controls, save, language.
Save conflict / recovery	Both versions, timestamps, summaries, preserve-copy choice.

Navigation rules

The harbor is the home screen. Secondary screens open as sheets or panels and return to the same camera position. Pause gameplay while a management panel is open by default. Back closes the current overlay before leaving the harbor. Avoid simultaneous modal stacks.

Main 3D screen and UI specification

Composition

Top left: profile and current harbor. Top center: coins and optional progression level. Top right: pause/settings. Left edge: tracked quest with one clear next step. Bottom left: mobile joystick. Bottom right: action button and carry indicator. Bottom center: a small station context card shown only in range.

Visual system

Token	Specification
Primary ink	Deep navy #153449 for readable text and outlines.
Action color	Ocean teal #008F95 for primary actions and active stations.
Reward accent	Warm gold #E8AF4A for earned coins and completed goals.
Surfaces	Warm off-white panels with sufficient contrast and restrained shadows.
Typography	Plain sans-serif; scalable labels and tabular figures for quantities.
Touch targets	Target at least 48 dp on Android; equivalent usable CSS targets on web.
Feedback	Short collection pulses; gentle vibration option; captions and icons.

Functional screen states

Every screen defines initial loading, populated, empty, unavailable, error, and recovery states. A disabled upgrade explains the missing condition. A machine blockage links to its cause. A failed purchase preserves coins and offers a retry. Loading never shows a misleading zero balance.

Responsive layout

Landscape mobile uses compact sheets and larger controls. Desktop uses side panels with more detailed metrics. Small touch browsers retain safe-area padding and avoid browser gesture edges. Portrait can show a rotate suggestion plus accessible management screens; full portrait gameplay is a later UI adaptation.

Detailed screen acceptance criteria

Screen group	Acceptance criteria
Station detail	Selecting a cutter highlights it in 3D; counts match authoritative inventory; upgrade changes one committed level.
Workers	Current role and claimed task appear; reassignment cannot duplicate carried goods.
Warehouse	Available and reserved quantities are distinct; totals reconcile across containers.
Contracts	Submitting goods removes them once and grants one receipt; completed state persists.
Dashboard	Reports only committed sales; interval and gross/net labels are explicit.
Map	Switching harbor saves state; locked harbors show exact requirements.
Settings	Volume and graphics persist; reduced motion removes unnecessary camera effects.
Recovery	Corrupt latest save offers previous valid snapshot without overwriting both.

Tutorial script

1. Move to the pier. 2. Catch three fish. 3. Carry them to the cutter. 4. Wait for output. 5. Stock the counter. 6. Serve a customer at checkout. 7. Spend coins on capacity. 8. Hire a porter. Highlight one target at a time and advance based on state events, never animation timing alone.

Accessibility

Allow tap-to-move, adjustable joystick size, left/right layout options, captions for important alerts, distinct shape cues, scalable UI, and a reduced-effects preset. Optional timing bonuses have assistance. Avoid essential text embedded in world textures.

Android stack: Kotlin and native 3D

Primary mobile release: Android. Kotlin implements gameplay and platform integration. Jetpack Compose renders menus and HUD. Filament renders the world through an Android rendering surface. Official Filament samples include Kotlin glTF loading and Android surface integration [1, 2].

Layer	Chosen approach
Language / build	Kotlin, Gradle Kotlin DSL, Android Studio; pin compatible stable versions.
UI	Jetpack Compose, lifecycle-aware state, accessible semantics.
Rendering	Filament Android, gltfio asset loading, Kotlin scene adapters.
Simulation	Pure Kotlin modules; fixed timestep; no rendering dependency.
Persistence	Room or transactional local snapshot storage; DataStore for settings.
Networking	Ktor client with typed request/response serialization.
Audio	Native low-latency playback for short cues; streamed music.
Assets	Shared glTF/GLB meshes; Android-prepared textures and material variants.
Testing	Kotlin unit tests plus Android UI/lifecycle and device tests.

Renderer responsibility

Filament supplies rendering, not the complete game framework. Implement scene ownership, input, ground movement, navigation, animation state, transfers, task scheduling, and audio mapping in the app. Prototype animated characters, water, transparency, and Compose layering before committing content production.

Platform boundary

Native Android does not automatically produce an iOS build. An iOS release would need a separate host and rendering/UI integration, with potential Kotlin Multiplatform reuse evaluated later. The mobile contract in this blueprint is Android; the web build can serve other devices through compatible browsers.

Web stack: TypeScript and React

Layer	Chosen approach
Language / tooling	TypeScript in strict mode, React, Vite build pipeline.
3D rendering	Three.js WebGLRenderer using WebGL 2 [3].
UI styling	CSS or Tailwind tokens shared with the Android design specification.
Simulation	Pure TypeScript engine independent of React and Three.js.
Persistence	IndexedDB transactions for snapshots; local preferences for settings.
Data validation	JSON Schema validation at content import and save load.
Audio	Web Audio API with user-gesture unlock and muted startup handling.
Testing	Vitest simulation checks and Playwright browser journeys.
Delivery	Versioned static assets through a CDN; optional installable web shell.

Rendering boundary

A React game host owns canvas creation and teardown. The engine writes transforms and animation parameters through a presentation adapter. Keep per-frame state out of React; publish small HUD snapshots around 5-10 times per second and immediate transaction events for purchases and rewards.

Browser lifecycle

Pause when hidden; reconcile offline state on resume. Handle resize, device-pixel ratio, keyboard focus, audio unlock, GPU context loss, and tab eviction. Cap catch-up steps after a long stall. Unmount removes event handlers and disposes textures, geometry, and audio nodes.

Touch browser support

Reuse mobile control rules, not native UI code. Test viewport resizing from browser chrome, safe areas, gesture interference, and low-memory eviction. Require a supported WebGL 2 context and offer a clear diagnostic screen when unavailable.

Shared rules and cross-platform parity

Kotlin and TypeScript are separate runtime implementations. Share specifications, content files, assets, formulas, and fixture tests; do not assume direct source-code reuse. One team owns the canonical simulation contract and accepts changes only when both implementations pass the same fixtures.

Shared package	Contents
Content	Species, machines, recipes, upgrade curves, harbors, quests, workers.
Schema	Stable IDs, enum values, allowed ranges, units, save versions.
Assets	Source meshes, animations, textures, icons, and platform export manifests.
Fixtures	Initial state + commands + tick count + expected canonical state.
Localization	String IDs, translations, placeholders, pluralization rules.
Design tokens	Colors, spacing intent, type scale, semantic states.

Simulation contract

Use integer ticks and quantities. Start at 20 simulation ticks/second, with interpolated visuals. Define fixed update order: commands; movement; arrivals; transfers; production; customer payment; task scheduling; quest events; checkpoint requests. Stable entity ordering resolves ties.

Randomness and arithmetic

Specify one seeded PRNG algorithm and test vectors in both languages. Store money in coins and time in ticks. Use scaled integers for rates where feasible, with documented floor/rounding rules. Restrict persisted integers to a safe shared range. Do not promise visual physics determinism; parity covers canonical economy and task state.

Version policy

Save schema version and content version travel with every snapshot. New clients migrate supported old saves; old clients must refuse unknown newer schemas without erasing them. Hash the content manifest and use it to diagnose mismatched balance.

System architecture and module ownership

Module	Responsibility
Game core	Clock, IDs, commands, events, deterministic rules, invariants.
World / movement	Zones, routes, positions, collisions, reachability.
Fishing	Catch timers, zone stock, species selection, carry checks.
Inventory	Ownership, capacity, reservation, transfer transactions.
Production	Recipes, reserved input batches, output commits.
Customers	Demand, routing, stock reservation, queues, payment.
Workers	Task generation, leases, assignment, cancellations.
Progression	Quests, contracts, unlock graph, upgrades, harbor mastery.
Persistence	Snapshots, migrations, backups, offline receipts.
Presentation	Meshes, animation, sound, effects, UI snapshots.
Online adapter	Authentication, save revisioning, entitlements, optional events.

Event discipline

Commands express player intent. The simulation validates them and emits results. UI effects consume events but never change economic state. Use correlation IDs for purchases, batches, sales, contract completion, and offline claims. Maintain a small debug ledger to explain discrepancies.

Concurrency

Keep canonical simulation mutations on one serialized loop. Background work may decode assets, perform networking, or serialize an immutable snapshot. Apply remote results only through validated commands. A save must represent one consistent tick, not a mix of adjacent systems.

Data model and example content

Entity	Key fields
PlayerProfile	ID, settings, unlocks, globalCoins, activeHarbor, schemaVersion.
HarborState	ID, plots, stations, inventories, workers, quests, accountedThrough.
Inventory	ID, capacityUnits, quantities, reservations, version.
MachineState	ID, definitionId, level, inputId, outputId, activeBatch.
WorkerState	ID, role, assignment, carryInventoryId, taskLease.
CustomerState	ID, order, basketId, queueSlot, patienceTicks, paidSaleId.
Receipt	Unique ID, type, delta, priorRevision, resultingRevision, tick.

Recipe example

ID: recipe.basic_portions. Machine: machine.basic_cutter. Input: species.silver_sprat, quantity 1. Output: product.basic_portion, quantity 2. Duration: 100 ticks at 20 Hz. Output unit price: 5 coins. Level-one input capacity: 10 fish. Output capacity: 20 portions.

Upgrade example

ID: upgrade.cutter_speed_2. Target machine level: 2. Cost: 300 coins. New batch duration: 80 ticks. Requirement: quest.first_ten_sales. Apply the new duration to the next batch; finish an active batch under its existing timer. Store this policy in the simulation specification.

Required invariants

No negative inventory, duplicate ownership, over-capacity committed stock, duplicate payment, or references to missing IDs. Every reserved item belongs to a valid claimant. Every completed batch has a unique output commit. A plot cannot contain overlapping blocking footprints. Check these in development builds and fixture tests.

Saving, cloud sync, and accounts

Local-first save strategy

Maintain latest and previous valid snapshots. Save after major purchases, harbor transitions, contract submission, and offline settlement; periodically checkpoint ordinary progress. Serialize the canonical state plus version and checksum. A checksum catches accidental corruption, not deliberate editing.

Account behavior

Guest play is first-class. Optional sign-in attaches a selected local save to an account after displaying the consequence. Account deletion and local reset are separate actions. Keep gameplay available when a server is down.

Endpoint example	Contract
GET /profile	Return owned profile metadata and latest cloud revision.
GET /saves/{harborId}	Return a versioned snapshot and manifest identity.
PUT /saves/{harborId}	Require expected revision and idempotency key.
POST /offline-claims	Online authoritative elapsed-time claim with one receipt.
POST /purchase-events	Server verifies provider evidence and grants entitlement once.
DELETE /profile	Authenticated deletion with documented retention outcome.

Conflict handling

Use optimistic concurrency. On conflict, preserve both snapshots and present coins, harbor progress, playtime, and timestamps. Never merge coin balances by addition. Choosing one branch produces a new revision while retaining a recovery copy. Keep API secrets outside clients.

Security scope

Optional Kotlin/Ktor services and PostgreSQL can support accounts and saves. Validate ownership, payload sizes, rate limits, and purchase evidence. Offline single-player saves can be edited; competitive leaderboards would need a separately designed authoritative system.

3D assets, animation, and sound

Art direction

Use premium stylized 3D: rounded machinery, appealing materials, clear silhouettes, rich water color, warm dock lighting, and readable product stacks. Prioritize animation and material consistency over photorealism. Fish processing is clean and non-graphic.

Asset family	Production scope
Characters	Player plus modular worker/customer variants; shared rigs where possible.
Fish / products	Distinct species models, portion packs, boxes, bulk cartons.
Stations	Machines, counters, checkout, storage, upgrades, plot markers.
Environment	Pier modules, boats, water, buildings, props, signs, lighting.
Animation	Idle, walk, cast, retrieve, carry, transfer, operate, pay, celebrate.
UI / audio	Consistent icons, native/vector labels, fishing cues, machinery, coins, ambience.

Export contract

Use one world scale, pivot convention, axis conversion, named sockets, and clip names. Validate each GLB in both renderers. Materials, transparency, shadows, and texture formats need platform-specific checks even when geometry is shared. Build a manifest with file hashes and compatible versions.

Performance art rules

Use pooled visual items, low-detail distant characters, instanced repeat props, baked ambient lighting, and simple water shaders. Carried stock follows attachment points; it does not create dozens of physics bodies. Reduce transparent layers and particle overdraw on low presets.

Audio

Layer gentle harbor ambience, machine loops, footsteps, catch cues, and purchase feedback. Cap concurrent voices and avoid repetitive loud coin effects. Provide independent music, effects, and haptics settings. Important production alerts also have visible labels.

Performance and platform quality

These are proposed budgets to measure on a named device matrix during the prototype. They are not verified performance claims.

Area	Initial target
Android frame rate	Stable 30 FPS baseline; optional 60 FPS on capable devices.
Web frame rate	60 FPS at 1080p on a chosen midrange desktop; 30 FPS low preset.
Simulation	20 Hz fixed step with limited catch-up work.
Population	Start with 1 player, 8 workers, and 24 visible customers.
Draw calls	Target below 200 typical on mobile and 350 on desktop; profile actual GPU cost.
Texture budget	Start around 128 MB resident mobile texture allocation; validate device limits.
First playable web	Target less than 20 MB compressed initial content, then load harbors on demand.
Stability	No sustained memory growth over repeated harbor load/unload cycles.

Native tests

Background/foreground, process death, rotation policy, interruptions, audio focus, thermal throttling, low-memory devices, save migrations, touch target reach, and surface teardown. Keep heavy decoding off the UI thread while respecting renderer threading rules.

Browser tests

Supported Chromium, Firefox, and Safari configurations; WebGL 2 availability; keyboard/touch; resize; context loss; tab eviction; storage unavailability; audio gesture restrictions; network interruption. Show useful retry and recovery screens.

Quality scaling

Scale resolution, shadow quality, effects, distant NPC detail, and water independently. Preserve interaction zones and economic rules across all presets. Visual quality changes must never alter catch results or production rates.

Monetization, analytics, and operations

Business model proposal

Launch a free starter harbor and offer a one-time full-game unlock as the initial model to test. Keep optional cosmetics separate from production power. The final price requires commercial validation. Avoid setting different economic rules across platforms merely because billing mechanisms differ.

Purchases

Restore valid entitlements, handle pending/cancelled purchases, and grant once after verification. Online purchase records must survive reinstall; guest purchases need a supported recovery association. Use the applicable platform billing mechanism and verify its current requirements before release.

Optional analytics

Event	Decision it supports
tutorial_step_completed	Identify where onboarding fails.
first_sale / first_upgrade	Check time to core reward and reinvestment.
machine_blocked_summary	Distinguish capacity, input, and transport bottlenecks.
harbor_unlocked	Measure progression pacing and drop-off.
offline_claimed	Check rewards, caps, and unintended repetition.
save_recovery / sync_conflict	Track reliability and cloud friction.
frame_time_summary	Find devices and scenes needing optimization.

Collect only what is needed, respect consent choices, avoid personal details in event payloads, and retain diagnostics for a defined period. Gameplay metrics should not contain account secrets or full save contents.

Live content

Distribute versioned manifests with rollback support. Do not change recipe output or upgrade costs midway through an active batch without a migration rule. A malformed remote manifest falls back to the last valid local content.

Production phases and acceptance gates

Phase	Deliverable and exit condition
1. Cross-platform spike	One rigged character, water, machine, UI overlay, and save load on Android and web.
2. Core loop	Catch → transfer → process → stock → checkout → upgrade works on both platforms.
3. Automation	Workers and customer reservations remain consistent under congestion.
4. Vertical slice	One complete harbor, onboarding, progression, offline claim, and recovery.
5. Tooling	Schema validation, asset checks, balance fixtures, and parity testing.
6. First release	Two harbors, eight species, four machine classes, four worker roles, core screens.
7. Complete product	Five harbors, about twenty species, six machine classes, contracts, mastery, optional cloud saves.
8. Release hardening	Device matrix, migrations, entitlements, accessibility, and performance gates.

Team responsibilities

Product/game designer owns pacing and content rules. Kotlin engineer owns native gameplay and integrations. Web engineer owns React and browser runtime. A 3D artist/technical artist owns asset parity and optimization. QA covers shared fixtures, economy, device lifecycle, and progression. Roles may be combined, but ownership must remain clear.

Definition of done

Both clients pass canonical fixtures; shipped objectives are reachable; inventories reconcile; rewards are idempotent; latest and backup saves restore; required screens handle errors; supported devices meet the agreed budget; all assets have documented usage rights. Estimate calendar duration after the vertical slice demonstrates actual production throughput.

Verification scenarios and implementation backlog

Scenario	Expected result
Reload during cutting	Input remains reserved; output appears exactly once after remaining duration.
Two porters target same stock	Reservations prevent double pickup and release correctly on cancellation.
Customer leaves queue	Unpaid basket follows return policy; no revenue is recorded.
App closes during purchase	Restore either before or after the committed upgrade; never deduct twice.
Offline claim retried	Same receipt and balance; no second grant.
Device clock moves backward	No negative reward or corrupted accounted-through time.
Cloud conflict	Both versions are preserved and no balances are added together.
Old save loads	Supported migration preserves owned items and upgrade levels.
Station output full	Production blocks visibly without losing reserved ingredients.
Android/web same fixture	Canonical inventories, coins, batches, and quest states match.

First implementation backlog

Define units and schemas; create example content pack; implement inventory and receipts; build fixed clock; implement movement and zones; add fishing; implement batch production; create customer checkout; add upgrade command; implement worker leases; build saves; add UI states; validate parity; profile real devices. Deliver a playable slice before adding the remaining harbors.

Unresolved decisions with safe defaults

Project title is Fish Shopping. Android and browser are release platforms; iOS is separate future scope. Landscape is the gameplay orientation. No real-time spoilage at launch. Local play works without sign-in. Monetization begins with a free starter area and optional full unlock. These defaults let engineering proceed without blocking core development.

Official references and handoff notes

Technical selection combines user-requested languages with rendering and UI libraries. Android and web have distinct renderer implementations; sharing art files does not guarantee identical materials or performance. Validate representative assets on both before scaling content.

Official sources checked November 5, 2026

[1] Filament Android samples. Surface integration and glTF demonstration projects.
google.github.io/filament/dup/android_samples.html

[2] Filament Kotlin glTF sample and engine repository. Android bindings and asset-loading integration.
github.com/google/filament
Kotlin glTF viewer source in the official repository

[3] Three.js WebGLRenderer documentation. The renderer uses WebGL 2.
threejs.org/docs/pages/WebGLRenderer

[4] Android UI direction and Compose. Kotlin-based UI guidance.
android-developers.googleblog.com/2026/05/android-ui-development-is-compose-first.html

Handoff package to produce during development

Versioned content schemas and examples; reference economy spreadsheet; Android and web repositories; canonical test fixtures; asset source files and exports; interaction specifications; save migration fixtures; device performance results; release checklist; and ownership/license records. This PDF is the governing product specification until replaced by an explicitly approved revision.

Product promise: a real, fully 3D fishing-to-market business that feels responsive to control, satisfying to automate, and consistent across native Android and web.